JavaScript Fresher Interview Preparation

Notes Covered So Far

These notes cover the concepts studied so far: JavaScript basics, variables, data types, operators, type coercion, equality, truthy/falsy values, functions, scope, hoisting, closures, and the **this** keyword.

1. JavaScript Basics

JavaScript is a high-level, dynamically typed programming language mainly used to make web pages interactive and dynamic. It runs in browsers and can also run on servers using environments such as Node.js.

HTML → structure **CSS** → appearance **JavaScript** → behavior/logic

2. Variables: var, let and const

var is function-scoped. **let** and **const** are block-scoped.

Feature	var	let	const
Scope	Function	Block	Block
Redeclaration	Yes	No	No
Reassignment	Yes	Yes	No
Hoisting behavior	Initialized as undefined	TDZ	TDZ

Important: const prevents reassignment of the variable/reference, but object properties can still be modified.

```
const person = { name: "Rahul" };
person.name = "Amit"; // allowed
```

3. Data Types

JavaScript data types are divided into **primitive** and **non-primitive** types.

Primitive Type	Example / Meaning
Number	25, 10.5 — integers and decimals use the Number type
String	"Hello" — text
Boolean	true / false
Undefined	Declared but generally not assigned a value
Null	Intentional absence of a value
BigInt	Very large integers, e.g. 123n
Symbol	Unique identifiers
Object	Non-primitive; includes objects and arrays

typeof can be used to check a type. Important JavaScript quirks: **typeof null** returns "object", and arrays also return "object" with typeof.

4. Operators

Arithmetic: +, -, *, /, %, **
Assignment: =, +=, -=, *=, /=
Comparison: >, <, >=, <=, ==, ===, !=, !==
Logical: && (AND), || (OR), ! (NOT)
Increment/decrement: ++, --

The modulus operator % gives the remainder. Example: 10 % 3 → 1.

5. Type Coercion

Type coercion is conversion of one data type into another during an operation. JavaScript can perform this conversion automatically.

```
10 + "5" → "105"
10 - "5" → 5
"10" * "2" → 20
```

The **+** operator is special because it can perform string concatenation. Operators such as -, *, and / generally convert numeric strings to numbers when possible.

Explicit conversion: Number(x), String(x), parseInt(x), parseFloat(x)

6. == vs ===

== performs loose equality and may perform type coercion.
=== performs strict equality and checks both value and type without implicit type conversion.

```
5 == "5" → true
5 === "5" → false
```

For modern JavaScript, prefer === unless there is a specific reason to use ==.

7. Truthy and Falsy Values

JavaScript treats values as truthy or falsy when used in conditions. Important falsy values include:

```
false, 0, -0, "", null, undefined, NaN
```

Important interview trap: **[]** and **{}** are truthy.

8. Functions

A **function** is a reusable block of code designed to perform a particular task.

```
Function declaration:
function add(a, b) { return a + b; }
```

Parameters are variables defined in the function declaration. **Arguments** are actual values passed when calling the function.

return gives a value back to the caller; **console.log()** prints a value.

Function expression:

```
const add = function(a, b) { return a + b; };
```

Arrow function:

```
const add = (a, b) => a + b;
```

Arrow functions are common in modern JavaScript and support concise syntax and implicit returns.

9. Scope

Scope determines where a variable can be accessed.

Global scope: variables declared outside functions/blocks can be accessible from inner scopes.
Function scope: var is function-scoped.
Block scope: let and const are block-scoped.

Scope chain: JavaScript searches for a variable from the current scope outward through parent scopes.

Important rule: an inner scope can access an outer scope, but an outer scope cannot directly access variables declared only inside an inner scope.

10. Hoisting

Hoisting is JavaScript's behavior of processing declarations before executing code in a scope. JavaScript does not literally move the source code.

```
console.log(a);
var a = 10;

// Output: undefined
```

With var, the declaration is processed and initialized with undefined, while the assignment happens at its original position.

```
console.log(a);
let a = 10;

// ReferenceError
```

let and const are hoisted/created in their scope but cannot be accessed before initialization. The period before initialization is called the **Temporal Dead Zone (TDZ)**.

Function declarations are hoisted and can generally be called before their declaration appears in the source.

Function expressions and arrow functions stored in variables do not behave like hoisted function declarations.

11. Closures

A **closure** is a function together with access to its surrounding lexical environment. It allows a function to access variables from its outer scope even after the outer function has finished executing.

```
function outer() {
let x = 10;
return function() {
return x;
};
}

let result = outer();
console.log(result()); // 10
```

Closures are useful for maintaining state and creating a form of data privacy.

Example: a counter function can keep its own private count variable between calls. Separate calls to the outer function create separate closures with separate state.

12. Lexical Scope

Lexical scope means a function's access to variables is determined by where the function is defined in the source code, not where the function is called.

Closures are based on lexical scope.

13. The this Keyword

this is a special keyword whose value depends primarily on how a function is invoked.

```
const person = {
name: "Ayush",
```

```
greet: function() {
console.log(this.name);
}
};

person.greet(); // Ayush
```

When a regular function is called as object.method(), this generally refers to the object.

Regular function: this depends on the call site.
Arrow function: does not have its own this; it inherits this from its surrounding lexical scope.

This distinction is especially important for callbacks and asynchronous code.

14. call(), apply(), bind()

call() invokes a function immediately and allows you to specify this; arguments are passed individually.
apply() also invokes immediately; arguments are supplied as an array-like collection.
bind() returns a new function with this bound to the supplied object.

Shortcut to remember:
call/apply → **execute now**
bind → **create a function for later**

Quick Interview Revision Sheet

Concept	Key point
JavaScript	High-level, dynamically typed language for web behavior and more
var	Function-scoped; declaration hoisted and initialized as undefined
let	Block-scoped; TDZ before initialization
const	Block-scoped; cannot be reassigned
Primitive types	Number, String, Boolean, Undefined, Null, BigInt, Symbol
Object	Main non-primitive type; arrays are objects
Type coercion	Automatic conversion between types during operations
==	Loose equality; may coerce types
===	Strict equality; value + type
Falsy	false, 0, -0, '', null, undefined, NaN
Function	Reusable block of code
Scope	Determines where variables are accessible
Hoisting	Declarations processed before execution
TDZ	Period where let/const cannot be accessed before initialization
Closure	Function retains access to surrounding lexical environment
this	Determined by how a regular function is called; arrow functions inherit it
call/apply/bind	Ways to control this for regular functions

Next topics: Arrays & Objects → map/filter/reduce/forEach → destructuring/spread/rest → Promises & async/await → Event Loop → DOM & Events → coding problems → mock interview.